



Lecture 6: Outputting Dynamic Values

INTERVIEW QUESTION

[> CORE](#)

How do you output dynamic values and set attributes in Svelte markup?

You have a variable in your script and want its value to appear inside a heading, or drive an element's class. How does markup read your script?

In modern web development, hardcoding static values into HTML is rarely enough. Your user interface needs to reflect the live data flowing through your application in real-time.

The Problem with Manual DOM Updates

Imagine you are building a shopping cart, and you need to display the current number of items. In vanilla JavaScript, you would define an empty placeholder in your HTML: `<span id="cart-count"></span>`. Then, in your JavaScript code, you write `document.getElementById('cart-count').innerText = count`. If the cart is empty, you might want to disable the checkout button by writing `document.getElementById('checkout-btn').disabled = true`.

You are constantly writing imperative code to reach across the boundary between HTML and JavaScript, manually gluing data into place. This is tedious to write, fragile to maintain, and easy to break if the DOM structure changes.

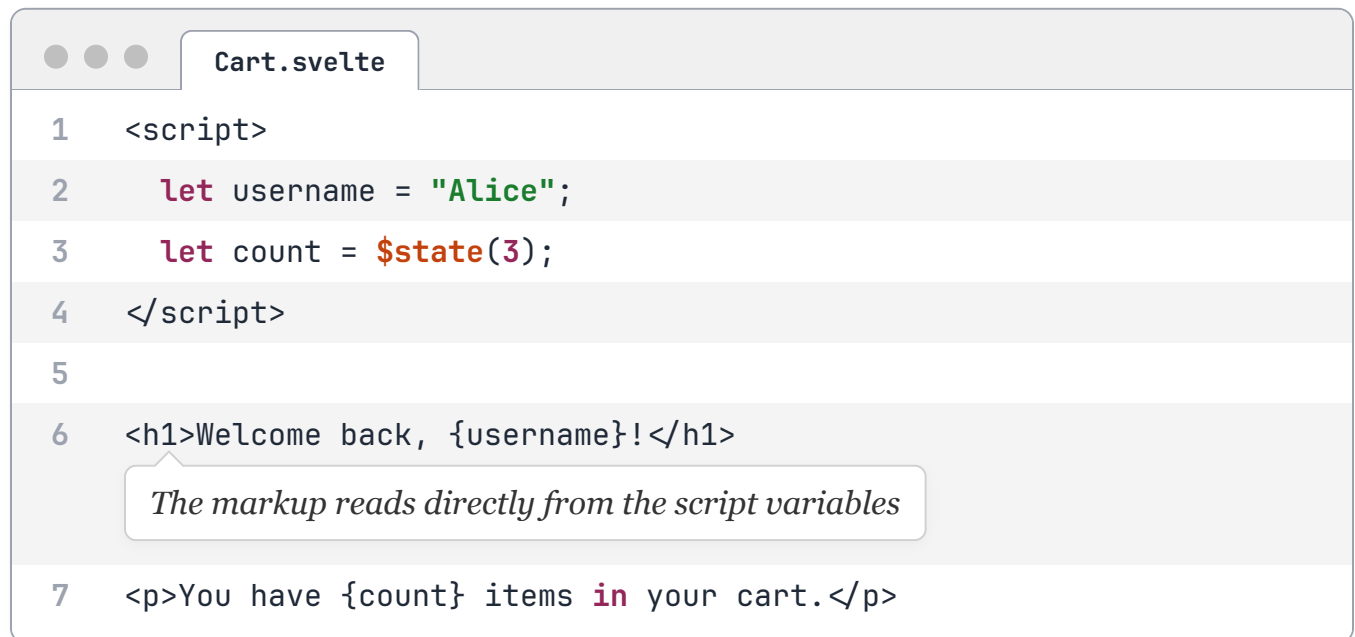
The Curly Braces Portal

Svelte solves this by allowing your markup to read directly from your script. By wrapping any JavaScript expression in **curly braces** (`{ }`), you open a portal into your `<script>` block right in the middle of your HTML.

Whenever the reactive value inside the braces changes, Svelte automatically updates the exact DOM node that relies on it.

Interpolating Text

To output a variable as text, just place the curly braces inside an HTML element. Svelte will evaluate the expression, convert the result to a string, and safely inject it as text (automatically escaping it to protect you from XSS injection attacks).



The screenshot shows a code editor window titled "Cart.svelte". The code is as follows:

```
1 <script>
2   let username = "Alice";
3   let count = $state(3);
4 </script>
5
6 <h1>Welcome back, {username}!</h1>
7
8 <p>You have {count} items in your cart.</p>
```

A callout box points to the `{username}` in line 6, containing the text: *The markup reads directly from the script variables*.

You can put any valid JavaScript expression inside the braces, not just variable names: `{count * 2}`, `{username.toUpperCase()}`, or `{count > 0 ? 'Full' : 'Empty'}`.

Dynamic Attributes

You can use the exact same curly brace syntax to set HTML attributes dynamically. Instead of a hardcoded string like `class="active"` , you pass the dynamic variable.



```
1 <script>
2   let isSubmitting = $state(false);
3   let themeColor = "blue";
4 </script>
5
6 <button disabled={isSubmitting} class={themeColor}>
  The disabled attribute and class are driven by variables
7   Checkout
8 </button>
```

KEY TAKEAWAY

If a boolean attribute (like `disabled` or `checked`) receives a truthy value, Svelte includes it. If it receives a falsy value, Svelte removes it completely from the DOM.

The Attribute Shorthand

Developers frequently name their variables identically to the attributes they want to control. For example, controlling an `id` with a variable named `id`, or `disabled` with a variable named `disabled`.

When the attribute name and the variable name match perfectly, Svelte provides a convenient shorthand: you can just write `{name}` instead of `name={name}`.



```
1 <script>
2   let disabled = $state(true);
3   let value = "Hello";
4 </script>
5
6 <input {disabled} {value} />
```

The shorthand `{disabled}` is identical to `disabled={disabled}`

INTERVIEW TIP

To impress the interviewer: Mention that unlike some frameworks that require special prefixes to bind attributes (like `v-bind:class` or `:class`), Svelte's curly braces work exactly the same way whether you are outputting text content or setting attribute values. It's a single, unified mental model.

Summary

To bridge the gap between your script and your markup, Svelte uses curly braces `{}` . This syntax evaluates JavaScript expressions and automatically updates the DOM when the underlying state changes. You can use curly braces to output text between HTML tags, or to dynamically set the values of HTML attributes. When an attribute and its controlling variable share the exact same name, Svelte offers a clean `{attribute}` shorthand.

	MANUAL (Vanilla JS)	SVELTE (Template Syntax)
Outputting Text	<code>element.innerText = value</code>	<code><h1>{value}</h1></code>
Setting Attributes	<code>element.disabled = true</code>	<code><button disabled={true}></code>
Matching Shorthand	N/A	<code><button {disabled}></code>